

第四讲(续) 案例六和SVM引申出的问题

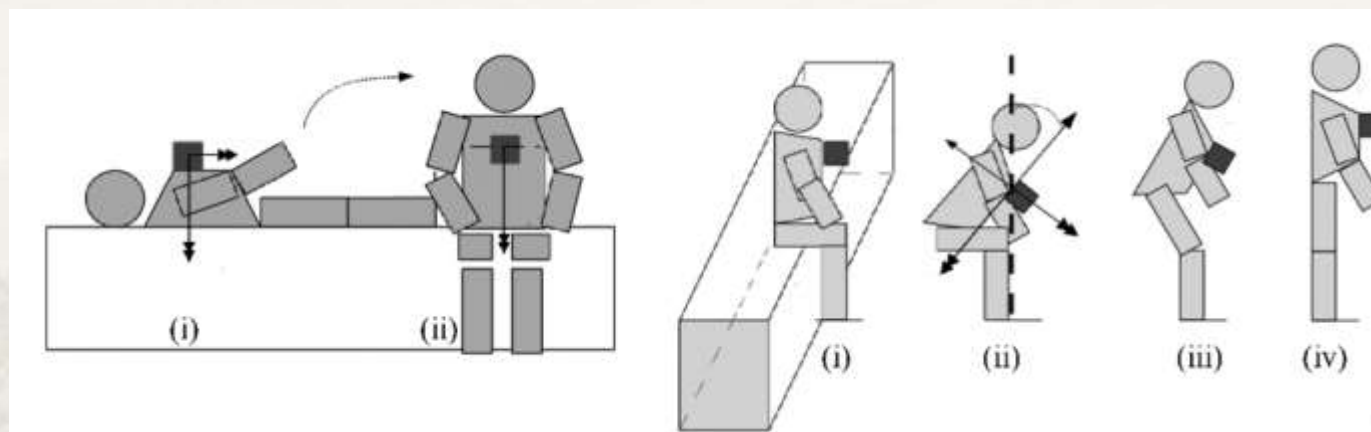
- 非平衡样本预测建模的一般策略
- 从优化到带正则项的回归

主要内容

- 讨论案例六的分类预测问题
 - 将案例简化为二分类问题，突出数据的非平衡特点
 - 非平衡数据建模的一般处理策略
- 讨论案例七：手写邮政编码的识别
 - 从优化到带正则项的回归
 - 岭回归和Lasso回归
 - Lasso-Logistic回归
 - 弹性网回归

案例六：老年人危险体位预警

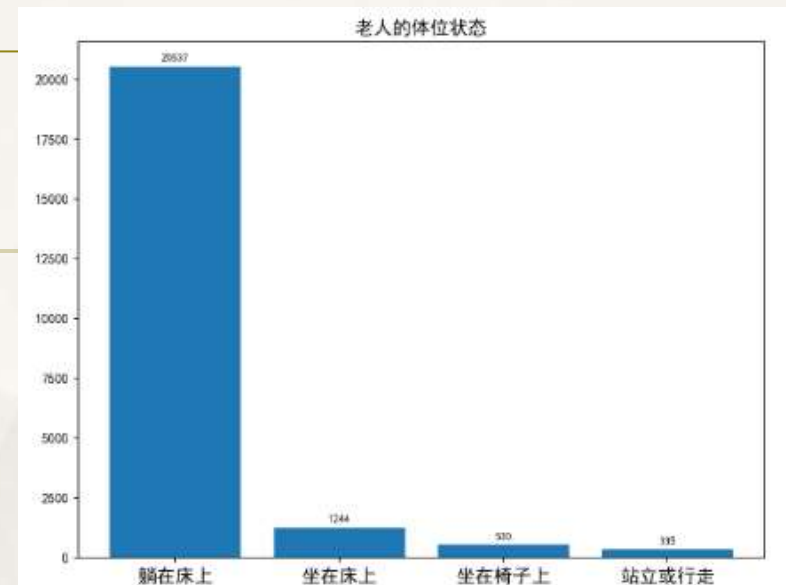
• 案例六简介



- 正方性表示无线穿戴设备
- 从平躺在床上，到坐起、起身站起、直立及行走过程中的体位 (Activity) 变化
- 随体位实时变化 (TimeStamp, 不同时间点)，无线穿戴设备的竖直高度 (vertical, 设备距地面的高度)、水平位置 (frontal, lateral, 设备距两个垂直墙体的距离) 数据以及倾角 (Phase) 等都会发生变化
- 数据通过房间内的3个射频识别 (Radio Frequency Identification, RFID) 设备实时采集并传回
 - 其中，2个RFID安装在天花板上，1个安装在墙上
- 老人在室内走动时各设备 (SensorID) 采集的信号强度 (RSSI) 会不同
- 数据采集时段内 (样本量22646)，老人们的体位状态包括
 - 1：坐在床上；2：坐在椅子上；3：躺在床上；4：站立或行走

- 简化案例六问题：安全体位和风险体位的**二分类**预测(仍采用SVM)

```
data['ActivityN'] = data['Activity'].map({3:0,1:0,2:1,4:1})
plt.subplot(1,2,2)
bars = plt.bar([1,2],data['ActivityN'].value_counts())
TextNumber(bars)
plt.title('老人的体位状态',fontsize = 14)
plt.xticks([1,2],['安全体位','风险体位'],fontsize = 14)
```



- 特点：典型的二分类**非平衡样本**

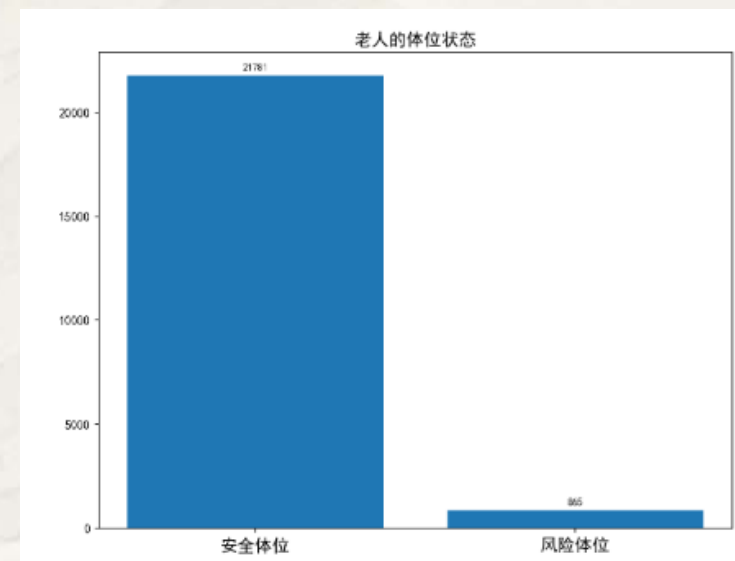
- 直接采用SVM预测：

```
1 y = data['ActivityN'].astype(int)
2 modelSVC = svm.SVC(kernel = 'rbf',C = 50)
3 modelSVC.fit(X,y)
4 print(classification_report(y,modelSVC.predict(X)))
```

- 问：**

- 预测效果是否理想？为什么？
- 可如何途径改善？

	precision	recall	f1-score	support
0	0.97	1.00	0.98	21781
1	0.93	0.16	0.27	865
accuracy			0.97	22646
macro avg	0.95	0.58	0.63	22646
weighted avg	0.97	0.97	0.96	22646



- 你认为**决策树**会受到样本非平衡性的影响吗？
- 还可以**如何改进**？

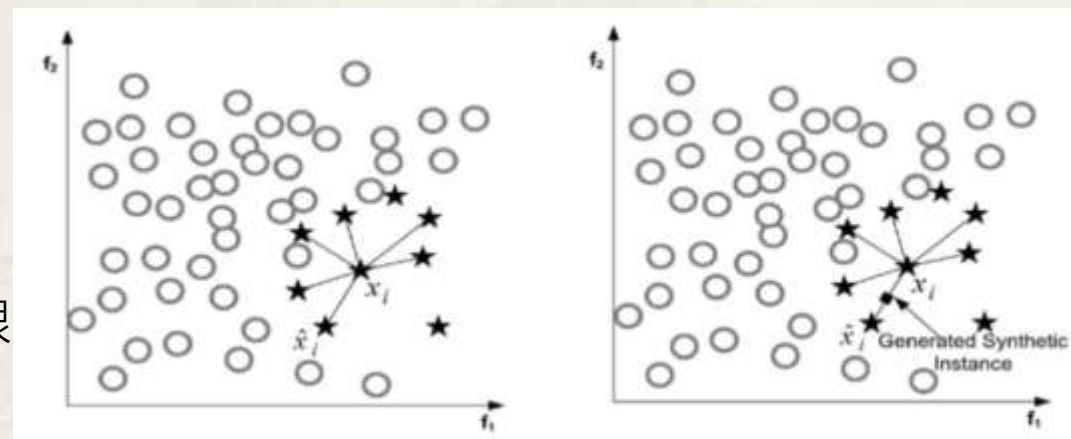
非平衡样本预测建模的一般策略

- **策略1**：改变样本的分布使两类样本量大致平衡
 - 随机欠采样(下采样)
 - 过采样(上采样)：随机过采样；SMOTE算法；自适应采样；等
- **SMOTE**(Synthetic Minority Over-sampling Technique)：对**每个少数类**观测从其k个同类近邻中随机抽取若干个，按以下策略合成少数类样本

$$x_{new} = x + rand(0,1) \times (\tilde{x} - x)$$

- **ADASYN**(Adaptive Synthetic Sampling)自适应采样：克服SMOTE算法均匀合成的不足，给每个**少数类**观测点 $\mathbf{x}_i$ 以不同的权重 w_i 并根据 w_i 决定为其合成 g_i 个少数类点： $g_i = w_i(N_M - N_L)$
 - N_M, N_L 分别为多数类和少数类的样本量
 - $n_i^{(k)}$ 是观测点 $\mathbf{x}_i$ 的k个近邻中多数类的个数
 - 合成策略同SMOTE
 - 更关注**分类边界附近**的观测点

$$w_i = \frac{n_i^{(k)}}{\sum_{j=1}^{N_L} n_j^{(k)}}$$



- 简化的案例六的分析：安全体位和风险体位的**二分类**预测(仍采用SVM)
 - 策略一**：改变样本的分布使两类样本量大致平衡

```
1 y = data['ActivityN'].astype(int)
2 modelSVC = svm.SVC(kernel = 'rbf', C = 50)
3 modelSVC.fit(X, y)
4 print(classification_report(y, modelSVC.predict(X)))
```

	precision	recall	f1-score	support
0	0.97	1.00	0.98	21781
1	0.93	0.16	0.27	865
accuracy			0.97	22646
macro avg	0.95	0.58	0.63	22646
weighted avg	0.97	0.97	0.96	22646

```
##pip install imbalanced-ensemble
from imbens.sampler import RandomOverSampler, SMOTE, ADASYN, RandomUnderSampler
from imbens.ensemble import BalanceCascadeClassifier
```

```
1 rus = RandomUnderSampler(random_state=123)
2 X_sample, y_sample = rus.fit_resample(X, y) #正负比例1:1
3 print('随机下采样的类别分布: %s; 样本量: %s' % (Counter(y_sample), len(y_sample)))
4
5 ros = RandomOverSampler(random_state=123)
6 X_sample, y_sample = ros.fit_resample(X, y) #正负比例1:1
7 print('随机上采样的类别分布: %s; 样本量: %s' % (Counter(y_sample), len(y_sample)))
8
9 smo = SMOTE(random_state=123, k_neighbors=5)
10 X_sample, y_sample = smo.fit_resample(X, y) #正负比例1:1
11 print('SMOTE采样的类别分布: %s; 样本量: %s' % (Counter(y_sample), len(y_sample)))
12
13 adas = ADASYN(random_state=123, n_neighbors=5) #如果一个少数类观测周围(5近邻)全
14 X_sample, y_sample = adas.fit_resample(X, y) #正负比例1:1
15 print('自适应采样的类别分布: %s; 样本量: %s' % (Counter(y_sample), len(y_sample)))
16
```

随机下采样的类别分布: Counter({0: 865, 1: 865}); 样本量: 1730

随机上采样的类别分布: Counter({0: 21781, 1: 21781}); 样本量: 43562

SMOTE采样的类别分布: Counter({0: 21781, 1: 21781}); 样本量: 43562

自适应采样的类别分布: Counter({1: 21853, 0: 21781}); 样本量: 43634

简化的案例六的分析：安全体位和风险体位的二分类预测(仍采用SVM)

- 策略一：改变样本的分布使两类样本量大致平衡
 - 采用自适应采样(重采样)处理后: 建立SVC二分类模型

```
1 y = data['ActivityN'].astype(int)
2 modelSVC = svm.SVC(kernel='rbf',C=50)
3 modelSVC.fit(X,y)
4 print(classification_report(y,modelSVC.predict(X)))
```

```
modelSVC = svm.SVC(kernel='rbf',C=50)
modelSVC.fit(X_sample,y_sample)
print('重采样数据集: \n',classification_report(y_sample,modelSVC.predict(X_sample)))
```

未做任何处理

	precision	recall	f1-score	support
0	0.97	1.00	0.98	21781
1	0.93	0.16	0.27	865
accuracy			0.97	22646
macro avg	0.95	0.58	0.63	22646
weighted avg	0.97	0.97	0.96	22646

重采样数据集:

	precision	recall	f1-score	support
0	1.00	0.95	0.98	21781
1	0.96	1.00	0.98	21853
accuracy			0.98	43634
macro avg	0.98	0.98	0.98	43634
weighted avg	0.98	0.98	0.98	43634

问:

- 你认为这样的非平衡处理策略有效吗?
 - 虚假评价/客观评价

```
print('原始数据集: \n',classification_report(y,modelSVC.predict(X)))
```

原始数据集:

	precision	recall	f1-score	support
0	1.00	0.95	0.98	21781
1	0.46	1.00	0.63	865
accuracy			0.96	22646
macro avg	0.73	0.98	0.80	22646
weighted avg	0.98	0.96	0.96	22646

以precision的下降为代价换取recall的提升,F1-得分提高!

- 简化的案例六的分析：安全体位和风险体位的**二分类预测** (仍采用SVM)
 - 采用自适应采样(**重采样**)处理后: 采用**简单的组合预测和投票策略**
 - 通过一组模型(base模型, 基学习器)以“投票”方式进行联合预测, 以提高预测效果

需估计泛化误差!

```
X_train, X_test, y_train, y_test = train_test_split(X, y, train_size = 0.90, random_state = 123)
adas = ADASYN(random_state=123, n_neighbors=5)
X_sample, y_sample = adas.fit_resample(X_train, y_train)
```

```
model1 = LM.LogisticRegression()
model2 = tree.DecisionTreeClassifier(max_depth=3, random_state = 123)
model3 = neighbors.KNeighborsClassifier(n_neighbors = 5, weights = 'distance')
model4 = GaussianNB()
model5 = svm.SVC(C = 50, kernel = 'rbf', probability = True)
estimators = [('SoftMax', model1), ('DT', model2), ('KNN', model3), ('NB', model3), ('SVC', model5)]

ModelEnsemble = ensemble.VotingClassifier(estimators, verbose = True, voting = 'hard')
#ensemble2 = VotingClassifier(estimators, voting='soft')
ModelEnsemble.fit(X_sample, y_sample)
```

```
[Voting] ..... (1 of 5) Processing SoftMax, total= 0.1s
[Voting] ..... (2 of 5) Processing DT, total= 0.0s
[Voting] ..... (3 of 5) Processing KNN, total= 0.0s
[Voting] ..... (4 of 5) Processing NB, total= 0.0s
[Voting] ..... (5 of 5) Processing SVC, total= 10.5s
```

```
VotingClassifier(estimators=[('SoftMax', LogisticRegression()),
                             ('DT',
                              DecisionTreeClassifier(max_depth=3,
                                                         random_state=123)),
                             ('KNN', KNeighborsClassifier(weights='distance')),
                             ('NB', KNeighborsClassifier(weights='distance')),
                             ('SVC', SVC(C=50, probability=True))],
                  verbose=True)
```


- 简化的案例六的分析：安全体位和风险体位的**二分类**预测(仍采用SVM)
 - 采用自适应采样(**重采样**)处理后:采用**简单的组合预测和投票策略**
 - 通过一组模型(base模型, 基学习器)以“投票”方式进行联合预测, 以提高预测效果

```
print('训练误差(重采样数据集)\n', classification_report(y_sample, ModelEnsemble.predict(X_sample)))  
print('训练误差(原始数据集)\n', classification_report(y_train, ModelEnsemble.predict(X_train)))  
print('测试误差\n', classification_report(y_test, ModelEnsemble.predict(X_test)))
```

自适应采样处理(全集)

原始数据集:

	precision	recall	f1-score	support
0	1.00	0.95	0.98	21781
1	0.46	1.00	0.63	865
accuracy			0.96	22646
macro avg	0.73	0.98	0.80	22646
weighted avg	0.98	0.96	0.96	22646

简单组合预测

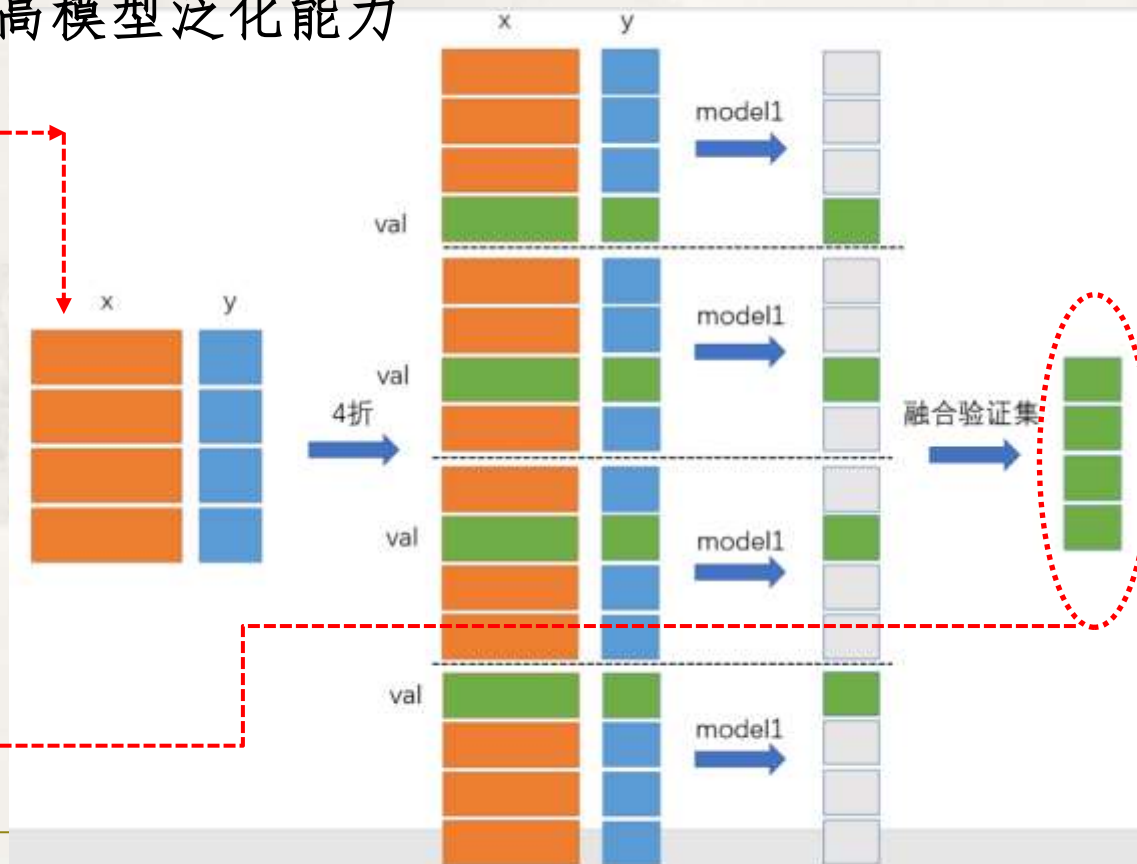
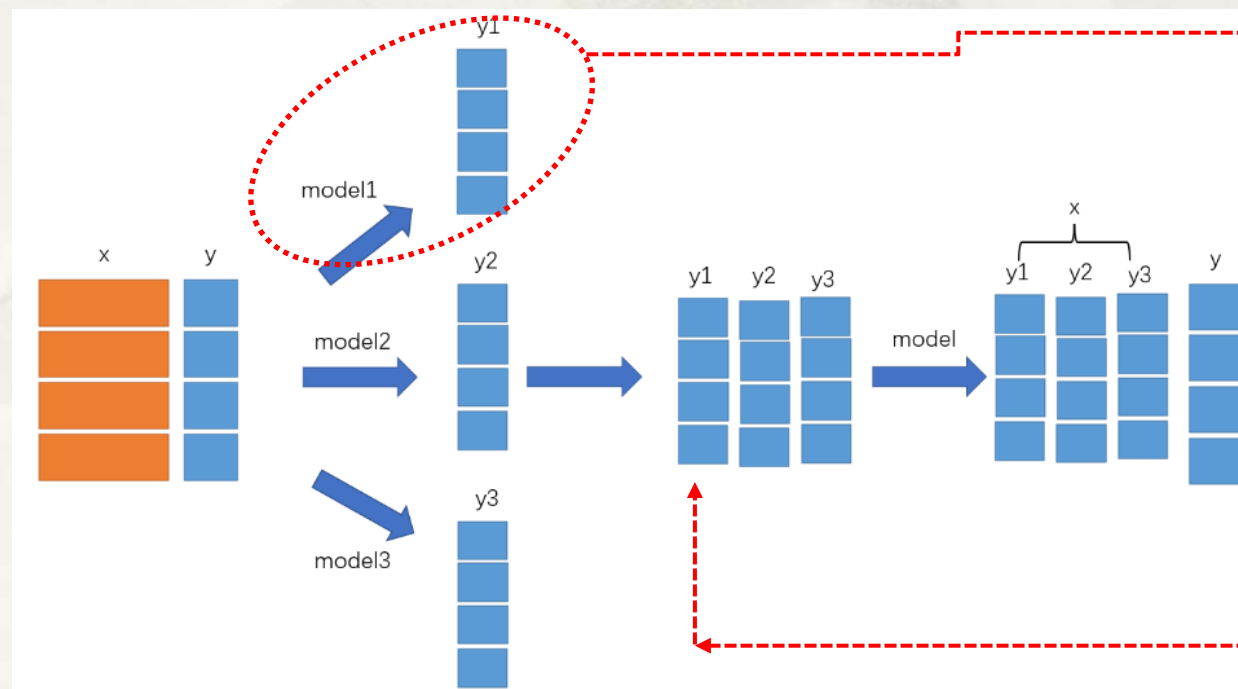
训练误差(重采样数据集)

	precision	recall	f1-score	support
0	1.00	0.95	0.98	19611
1	0.96	1.00	0.98	19587
accuracy			0.98	39198
macro avg	0.98	0.98	0.98	39198
weighted avg	0.98	0.98	0.98	39198

测试误差

	precision	recall	f1-score	support
0	1.00	0.96	0.98	2170
1	0.51	1.00	0.68	95
accuracy			0.96	2265
macro avg	0.76	0.98	0.83	2265
weighted avg	0.98	0.96	0.97	2265

- 简化的案例六的分析：安全体位和风险体位的**二分类**预测(仍采用SVM)
 - 采用自适应采样(**重采样**)处理后:采用**Stacking**(堆叠)策略
- Stacking(堆叠)策略:通常采用两层模型
 - 第一层base模型(基学习器)通常为**不同类型(异质)**的模型
 - 第一层输出作为第二层模型(元学习器)的输入,以最大限度提高预测精度
 - 可采用K-折交叉划分数数据集,避免过拟合提高模型泛化能力



- 简化的案例六的分析：安全体位和风险体位的二分类预测(仍采用SVM)
 - 采用自适应采样(重采样)处理后:采用Stacking(堆叠)策略

```
ModelEnsemble = ensemble.StackingClassifier(estimators=estimators, final_estimator=LM.LogisticRegression(), cv=5)
ModelEnsemble.fit(X_sample, y_sample)
```

```
print('训练误差(重采样数据集)\n', classification_report(y_sample, ModelEnsemble.predict(X_sample)))
print('训练误差(原始数据集)\n', classification_report(y_train, ModelEnsemble.predict(X_train)))
print('测试误差\n', classification_report(y_test, ModelEnsemble.predict(X_test)))
```

简单的组合预测

训练误差(重采样数据集)				
	precision	recall	f1-score	support
0	1.00	0.95	0.98	19611
1	0.96	1.00	0.98	19587
accuracy			0.98	39198
macro avg	0.98	0.98	0.98	39198
weighted avg	0.98	0.98	0.98	39198

测试误差				
	precision	recall	f1-score	support
0	1.00	0.96	0.98	2170
1	0.51	1.00	0.68	95
accuracy			0.96	2265
macro avg	0.76	0.98	0.83	2265
weighted avg	0.98	0.96	0.97	2265

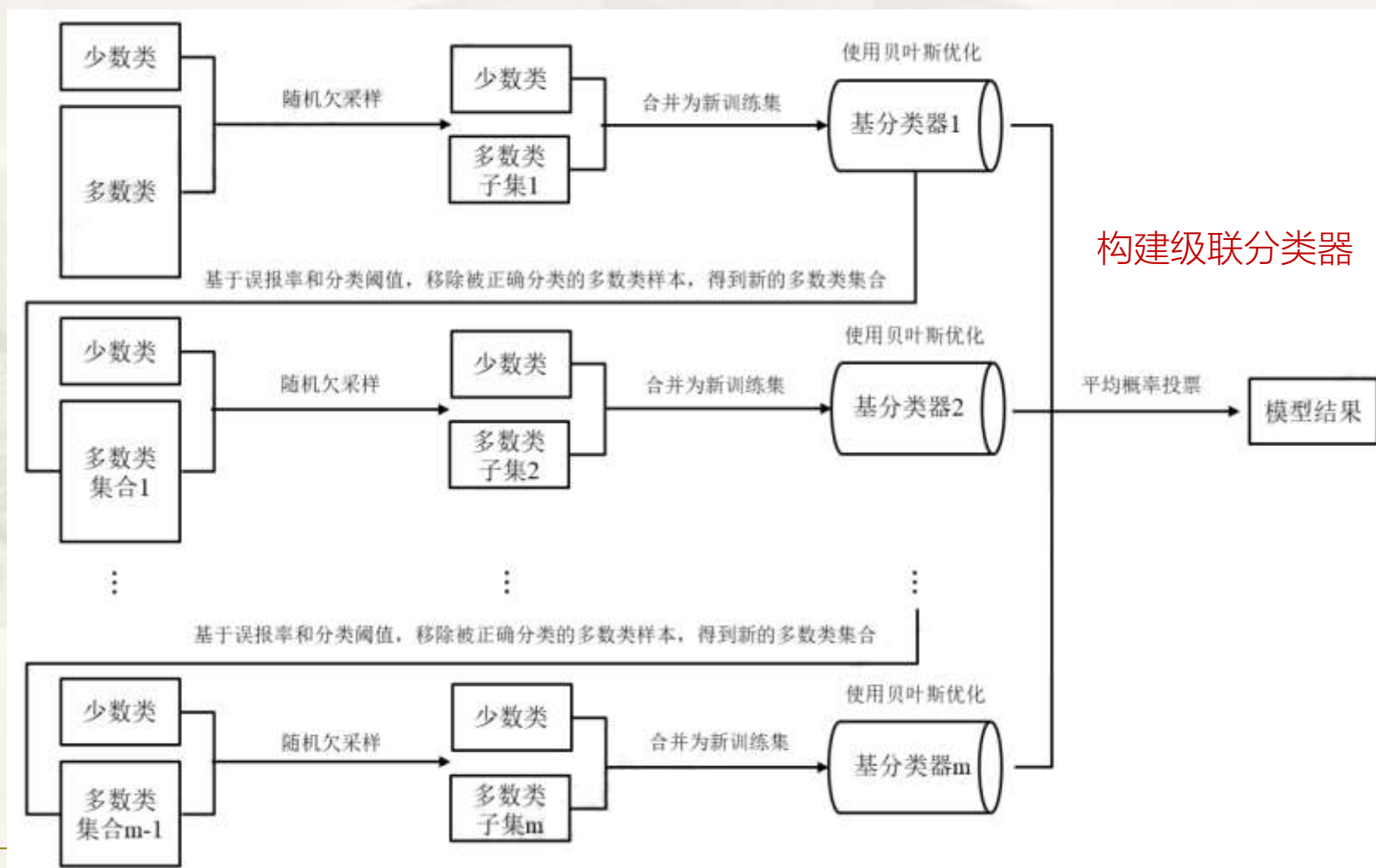
Stacking策略

训练误差(重采样数据集)				
	precision	recall	f1-score	support
0	1.00	1.00	1.00	19611
1	1.00	1.00	1.00	19587
accuracy			1.00	39198
macro avg	1.00	1.00	1.00	39198
weighted avg	1.00	1.00	1.00	39198

测试误差				
	precision	recall	f1-score	support
0	1.00	0.97	0.99	2170
1	0.60	0.95	0.74	95
accuracy			0.97	2265
macro avg	0.80	0.96	0.86	2265
weighted avg	0.98	0.97	0.97	2265

非平衡样本预测建模的一般策略

- 策略二： **BalanceCascade策略** (Liu等人(2009年), 处理不平衡数据集的组合分类预测算法)



- 简化的案例六的分析：安全体位和风险体位的**二分类**预测(仍采用SVM)
 - 采用**BalanceCascade策略**

```
##pip install imbalanced-ensemble
from imbens.sampler import RandomOverSampler, SMOTE, ADASYN, RandomUnderSampler
from imbens.ensemble import BalanceCascadeClassifier
```

```
1 BcClassifier = BalanceCascadeClassifier(estimator=svm.SVC(kernel='rbf',C=50,probability=True),
2                                     n_estimators=10, random_state=42)
3 BcClassifier.fit(X_train, y_train)
4 print('训练误差\n',classification_report(y_train,BcClassifier.predict(X_train)))
5 print('测试误差\n',classification_report(y_test,BcClassifier.predict(X_test)))
```

训练误差

	precision	recall	f1-score	support
0	1.00	0.96	0.98	19611
1	0.47	0.99	0.64	770
accuracy			0.96	20381
macro avg	0.73	0.97	0.81	20381
weighted avg	0.98	0.96	0.96	20381

测试误差

	precision	recall	f1-score	support
0	1.00	0.96	0.98	2170
1	0.52	1.00	0.68	95
accuracy			0.96	2265
macro avg	0.76	0.98	0.83	2265
weighted avg	0.98	0.96	0.97	2265

```
BcClassifier = BalanceCascadeClassifier(n_estimators=10, random_state=42) #默认是分类树
BcClassifier.fit(X_train, y_train)
print('训练误差\n',classification_report(y_train,BcClassifier.predict(X_train)))
print('测试误差\n',classification_report(y_test,BcClassifier.predict(X_test)))
print(BcClassifier.feature_importances_ )
```

训练误差

	precision	recall	f1-score	support
0	1.00	0.99	0.99	19611
1	0.74	1.00	0.85	770
accuracy			0.99	20381
macro avg	0.87	0.99	0.92	20381
weighted avg	0.99	0.99	0.99	20381

测试误差

	precision	recall	f1-score	support
0	1.00	0.98	0.99	2170
1	0.70	0.91	0.79	95
accuracy			0.98	2265
macro avg	0.85	0.94	0.89	2265
weighted avg	0.98	0.98	0.98	2265

[0.09704671 0.74154577 0.09350099 0.06790653]

可尝试不同的分类器

非平衡样本预测建模的一般策略

- 策略三：从损失函数角度进行优化--Focal Loss(焦点损失)

- 0-1分类的标准交叉熵损失：
$$L(y_i, P(\mathbf{X}_i)) = -[y_i \log P(\mathbf{X}_i) + (1 - y_i) \log(1 - P(\mathbf{X}_i))]$$

- Focal Loss是对标准交叉熵损失的改进：1类为少数类

- 调节因子 $(1 - P)^\gamma$ (通常 $\gamma = 2$)：保持难分样本损失基本不变的条件下，降低易分样本的损失贡献

$$FL = -y_i(1 - P)^\gamma \log(P) - (1 - y_i)P^\gamma \log(1 - P)$$

- 平衡因子 α (通常 $\alpha = 0.25$)：缓解样本不平衡性，避免过度放大由 γ 强调的难分样本的损失

$$FL = -y_i \alpha (1 - P)^\gamma \log(P) - (1 - y_i) (1 - \alpha) P^\gamma \log(1 - P)$$

- Focal Loss的简写形式：

$$FL(p_t) = -\alpha_t (1 - p_t)^\gamma \log(p_t) \quad p_t = \begin{cases} P, y_i = 1 \\ 1 - P, y_i = 0 \end{cases}$$

从优化到带正则项的回归

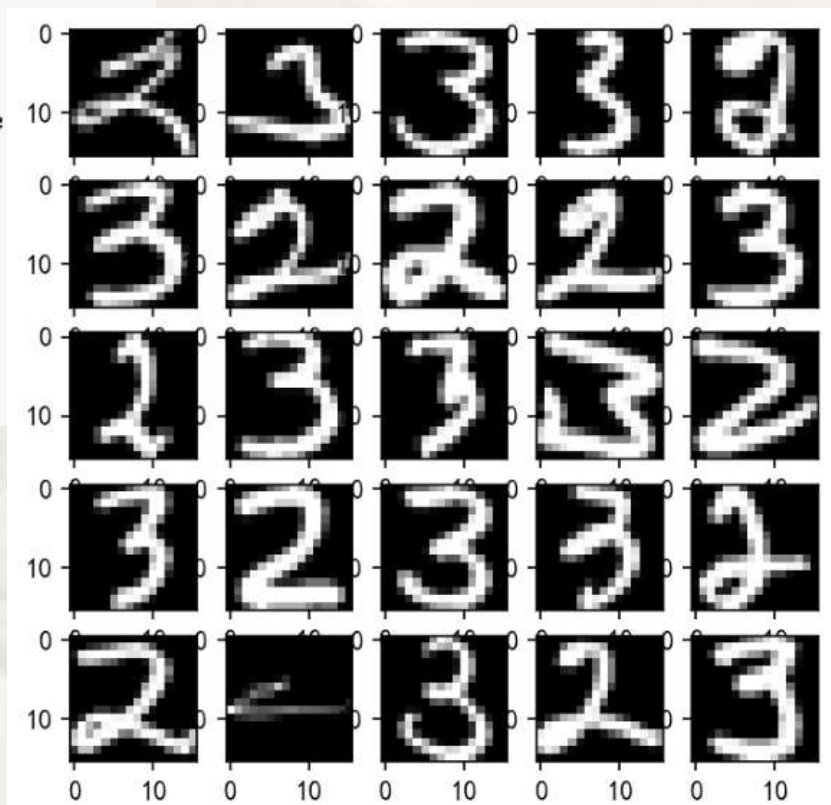
---嵌入式策略下的特征选择

- 岭回归和Lasso回归
- 弹性网回归

案例七：手写邮政编码的识别

- 图像为 16×16 点阵数据—维度较高
 - 利用**嵌入式**策略进行特征选择并建立**二分类**预测模型

```
data = pd.read_table('邮政编码数据.txt', sep = ' ', header = None)
tmp = data.loc[(data[0] == 2) | (data[0] == 3)]
X = tmp.iloc[:, 1:-1]; y = tmp.iloc[:, 0]
X_train, X_test, y_train, y_test = train_test_split(X, y, train_size = 0.70, random_state
np.random.seed(1)
ids = np.random.choice(len(y_train), 25)
plt.figure(figsize = (5, 5), dpi = 150)
for i, item in enumerate(ids, start = 1):
    img = np.array(X_train.iloc[item, :]).reshape((16, 16))
    plt.subplot(5, 5, i)
    plt.imshow(img, cmap = cm.gray)
```



- **问：**关注什么？
 - 什么是嵌入式特征选择
 - 基于嵌入式特征选择的回归预测问题
 - 基于嵌入式特征选择的分类预测问题

嵌入式策略下的特征选择

- 将特征选择过程嵌套在预测模型的训练中，既能够考察输入变量的重要性，也可同时给出最终的预测模型
 - 通过带**约束的预测建模**实现
- 经典方法：岭回归和Lasso回归

$$y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_p X_p$$

- 如果回归方程中 $X_1, X_2, \dots, X_p$ 均为重要性很高的变量： $\sum_{j=1}^p \beta_j^2$ 或 $\sum_{j=1}^p |\beta_j|$ 应很大
- 如果回归方程的 $X_1, X_2, \dots, X_p$ 均为不重要的变量： $\sum_{j=1}^p \beta_j^2$ 或 $\sum_{j=1}^p |\beta_j|$ 应很小
 - $\sum_{j=1}^p \beta_j^2$ 或 $\sum_{j=1}^p |\beta_j|$ 值会受 p 大小的影响
 - 为使大量不重要的变量不进入模型，可指定进入模型的所有变量的系数 $\sum_{j=1}^p \beta_j^2$ 或 $\sum_{j=1}^p |\beta_j|$ 小于某个阈值 s 即可，不能再增加了

- 岭回归和Lasso回归 $y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_p X_p$

目标函数：

$$\begin{aligned}\hat{\beta} &= \arg \min_{\beta} \sum_{i=1}^N L(y_i, \hat{y}_i) \\ &= \arg \min_{\beta} \sum_{i=1}^N (y_i - (\beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_p X_p))^2\end{aligned}$$

约束条件：

$$\sum_{j=1}^p \beta_j^2 \leq s \quad \text{或} \quad \sum_{i=1}^p |\beta_j| \leq s$$

- 以二元为例：

- 目标函数： $\arg \min_{\beta} \sum_{i=1}^N L(y_i, \hat{y}_i) = \arg \min_{\beta} \sum_{i=1}^N (y_i - (\beta_0 + \beta_1 X_1 + \beta_2 X_2))^2$

- 约束条件： $\sum_{j=1}^2 \beta_j^2 = \beta_1^2 + \beta_2^2 \leq s$, 或 $\sum_{j=1}^2 |\beta_j| = |\beta_1| + |\beta_2| \leq s$

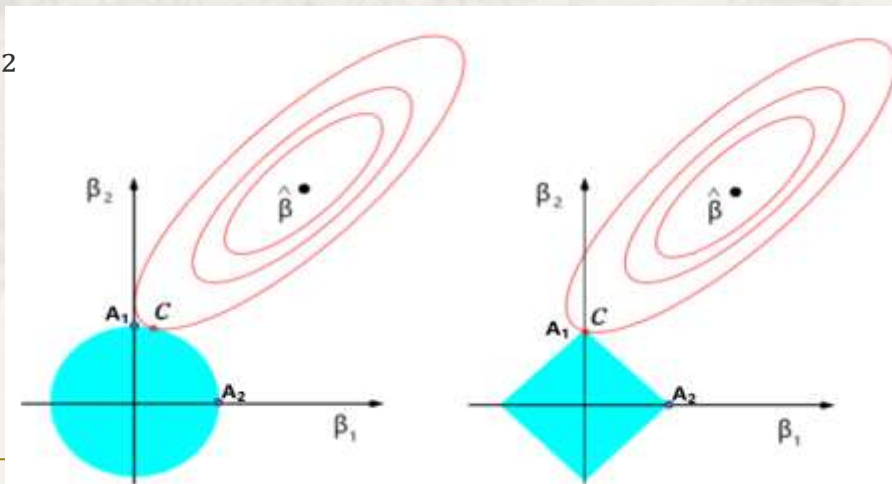
- 圆的标准方程： $(x - a)^2 + (y - b)^2 = (\sqrt{s})^2$

- 圆心坐标： $a=b=0$

- 菱形的标准方程： $|x-a|/h + |y-b|/k = 1$

- 菱形的中心点坐标： $a=b=0$

- 对角线长度的一半： $h=k=s$



参数 β 的最优解应在等高线与圆或菱形的相切点C处

- 嵌入式策略下的特征选择：

目标函数：

$$\begin{aligned}\hat{\beta} &= \arg \min_{\beta} \sum_{i=1}^N L(y_i, \hat{y}_i) \\ &= \arg \min_{\beta} \sum_{i=1}^N (y_i - (\beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_p X_p))^2\end{aligned}$$

约束条件：

$$\sum_{j=1}^p \beta_j^2 \leq s \quad \text{或} \quad \sum_{j=1}^p |\beta_j| \leq s$$

- 根据拉格朗日对偶性，等价表述为：

$$\text{obj}(\beta) = \sum_{i=1}^N \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j X_{ij} \right)^2 + \alpha \sum_{j=1}^p \beta_j^2$$

- 以上为岭回归 (Ridge Regression) 的目标函数

$$\text{obj}(\beta) = \sum_{i=1}^N \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j X_{ij} \right)^2 + \alpha \sum_{j=1}^p |\beta_j|$$

- 以上为Lasso (Least absolute shrinkage and selection operator) 回归的目标函数

- 第一项为损失函数：平方损失
- 第二项：度量了模型的复杂度
 - 称为模型的正则 (Regularization) 项
 - 分别为：L2正则化和L1正则化
 - $\alpha > 0$, 可调参数, 称为收缩参数
 - α 越大对应的约束参数 s 越小
 - α 越小对应的约束参数 s 越大

- 岭回归和Lasso回归
 - 求两者之和最小下的模型参数：

$$\hat{\beta}(\alpha) = \underset{\beta}{\operatorname{argmin}} \left[\sum_{i=1}^N \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j X_{ij} \right)^2 + \alpha \sum_{j=1}^p \beta_j^2 \right] \quad \hat{\beta}(\alpha) = \underset{\beta}{\operatorname{argmin}} \left[\sum_{i=1}^N \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j X_{ij} \right)^2 + \alpha \sum_{j=1}^p |\beta_j| \right]$$

- Python中的目标函数：

$$\hat{\beta}(\alpha) = \underset{\beta}{\operatorname{argmin}} \left[\frac{1}{2N} \sum_{i=1}^N \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j X_{ij} \right)^2 + \alpha \sum_{j=1}^p \beta_j^2 \right] \quad \hat{\beta}(\alpha) = \underset{\beta}{\operatorname{argmin}} \left[\frac{1}{2N} \sum_{i=1}^N \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j X_{ij} \right)^2 + \alpha \sum_{j=1}^p |\beta_j| \right]$$

- 收缩参数 α 本质是个模型**惩罚参数**，起到了对模型**复杂度的惩罚**作用
- α 较小极端情况下 $\alpha \rightarrow 0$:为最小化损失函数，模型复杂度最高，趋近普遍最小二乘解
- α 较大极端情况下 $\alpha \rightarrow \infty$:为最小化正则项，压缩 $\beta_j \rightarrow 0$ ，模型最简单
- α 起到了平衡模型训练误差和复杂度的作用

• Lasso回归: 探讨收缩参数 α 对回归系数和模型误差的影响

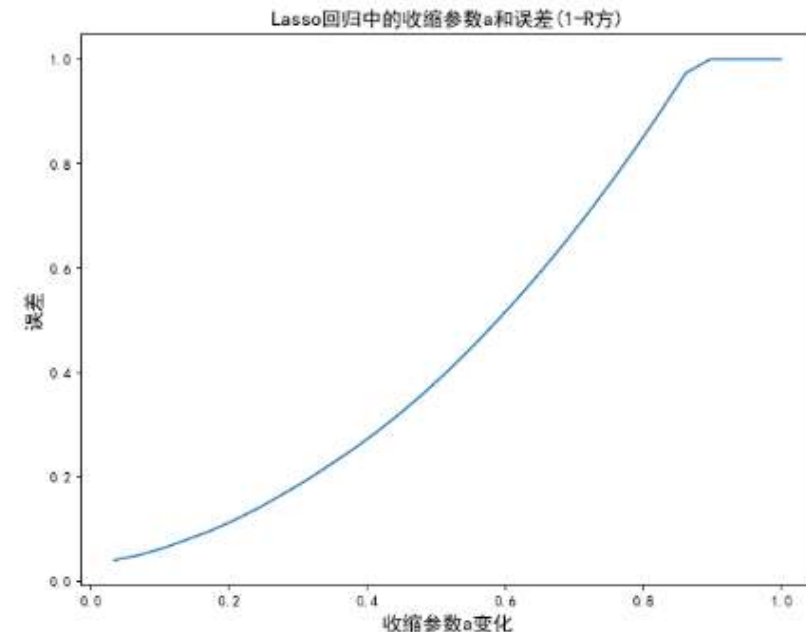
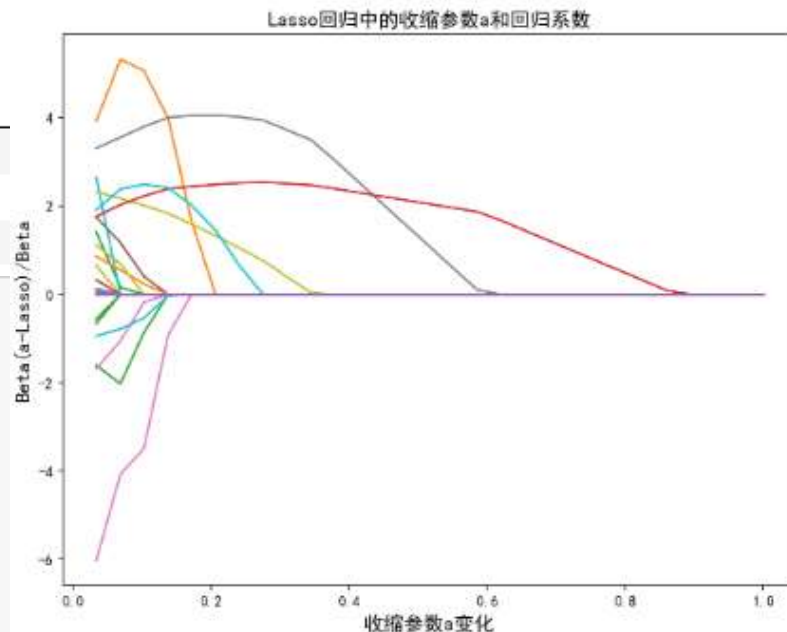
```
1 data = pd.read_table('特征提取.txt', sep = ',', header = None)
2 print(f'数据集的形状: {np.shape(data)}')
3 data
```

Chapter10-4.ipynb

数据集的形状: (1663, 257)

	0	1	2	3	4	5	6	7	8
0	2.989144	-1.0	-1.0	-1.0	-1.000	-1.000	-0.928	-0.204	0.751
1	3.009973	-1.0	-1.0	-1.0	-0.830	0.442	1.000	1.000	0.479
2	1.002830	-1.0	-1.0	-1.0	-1.000	-1.000	-1.000	-1.000	0.510

```
1 X = data.iloc[:, 1:-1]; y = data.iloc[:, 0]
2 alphas = list(np.round(np.linspace(0, 1, 30), 3))
3 coef = np.zeros((len(alphas), X.shape[1]))
4 err = []
5 for i, alpha in enumerate(alphas):
6     modelLasso = Lasso(alpha = alpha)
7     modelLasso.fit(X, y)
8     if i == 0:
9         coef[i] = modelLasso.coef_
10    else:
11        coef[i] = (modelLasso.coef_ / coef[0])
12    err.append(1 - modelLasso.score(X, y))
13 fig, axes = plt.subplots(nrows = 1, ncols = 2, figsize = (20, 7), dpi = 150)
14 for i in np.arange(0, X.shape[1]):
15     axes[0].plot(alphas[1:], coef[1:, i])
16 axes[0].set_title('Lasso回归中的收缩参数a和回归系数', fontsize = 14)
17 axes[0].set_xlabel('收缩参数a变化', fontsize = 14)
18 axes[0].set_ylabel('Beta(a-Lasso)/Beta', fontsize = 14)
19 axes[1].plot(alphas[1:], err[1:])
20 axes[1].set_title('Lasso回归中的收缩参数a和误差(1-R方)', fontsize = 14)
21 axes[1].set_xlabel('收缩参数a变化', fontsize = 14)
22 axes[1].set_ylabel('误差', fontsize = 14)
```



- Lasso中增大 α 会使更多系数被压缩严格为零
 - 稀疏性

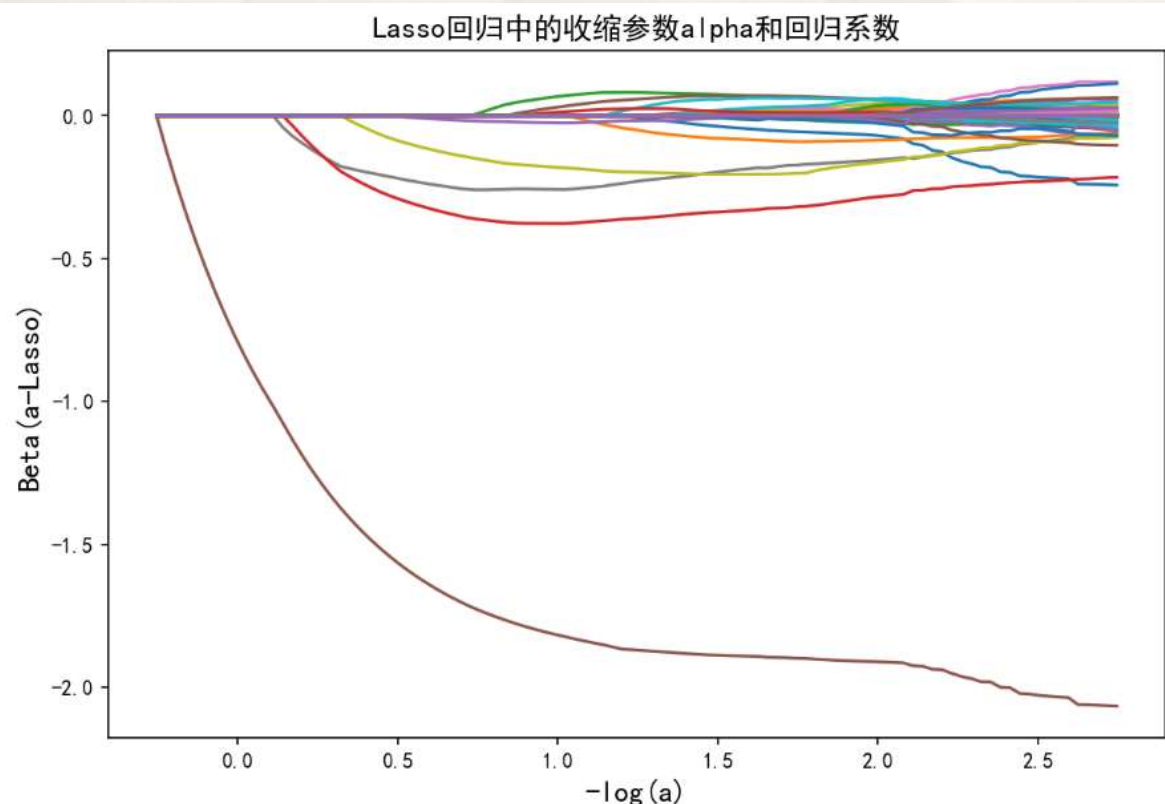
- Lasso回归: 探讨收缩参数 α 对回归系数和模型误差的影响

Chapter10-4.ipynb

```
1 fig = plt.figure(figsize=(9,6))
2 alphas_lasso, coefs_lasso, _ = lasso_path(X, Y)
3 print(alphas_lasso)
4 plt.plot(-np.log10(alphas_lasso), coefs_lasso.T)
5 plt.title('Lasso回归中的收缩参数alpha和回归系数', fontsize=14)
6 plt.xlabel('-log(a)', fontsize=14)
7 plt.ylabel('Beta(a-Lasso)', fontsize=14)
```

收缩参数 α 默认从大到小取100个值

```
[1.79120355 1.67048242 1.5578975 1.45290042 1.3549798 1.26365869
1.17849233 1.09906589 1.02499253 0.95591146 0.89148623 0.83140304
0.77536926 0.72311196 0.67437664 0.62892591 0.58653841 0.54700768
0.5101412 0.47575939 0.4436948 0.41379125 0.38590311 0.35989453
0.33563884 0.31301791 0.29192155 0.27224701 0.25389848 0.23678657
0.22082795 0.20594488 0.19206489 0.17912035 0.16704824 0.15578975
0.14529004 0.13549798 0.12636587 0.11784923 0.10990659 0.10249925
0.09559115 0.08914862 0.0831403 0.07753693 0.0723112 0.06743766
0.06289259 0.05865384 0.05470077 0.05101412 0.04757594 0.04436948
0.04137913 0.03859031 0.03598945 0.03356388 0.03130179 0.02919215
0.0272247 0.02538985 0.02367866 0.02208279 0.02059449 0.01920649
0.01791204 0.01670482 0.01557898 0.014529 0.0135498 0.01263659
0.01178492 0.01099066 0.01024993 0.00955911 0.00891486 0.00831403
0.00775369 0.00723112 0.00674377 0.00628926 0.00586538 0.00547008
0.00510141 0.00475759 0.00443695 0.00413791 0.00385903 0.00359895
0.00335639 0.00313018 0.00291922 0.00272247 0.00253898 0.00236787
0.00220828 0.00205945 0.00192065 0.0017912 ]
```



• 基于Lasso回归筛选重要变量

```

1 modelLasso = LassoCV()      默认5折      Chapter10-4.ipynb
2 modelLasso.fit(X,y)
3 print('Lasso剔除的变量:%d'%sum(modelLasso.coef_ == 0))
4 print('Lasso的最优收缩参数a: %.4f'%modelLasso.alpha_)
5 estimator = Lasso(alpha = modelLasso.alpha_)
6 selector = SelectFromModel(estimator = estimator)
7 selector.fit(X,y)
8 print('阈值: %s'%selector.threshold_)
9 print('保留的特征个数: %d'%len(selector.get_support(indices = True)))
10 print('lasso误差: %.2f'%(1 - modelLasso.score(X,y)))

```

Lasso剔除的变量: 153

Lasso的最优收缩参数a: 0.0012

阈值: 1e-05

保留的特征个数: 102

lasso误差: 0.02

Selector的阈值默认为特征重要性的均值，当模型带有L1/L2范数约束时默认为1e-05

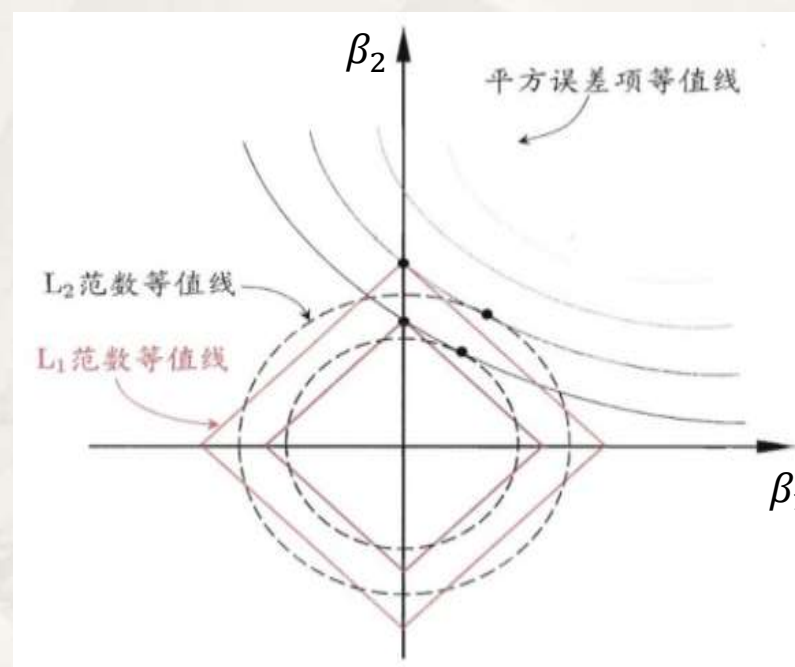
• Lasso回归和岭回归的比较

```

1 modelRidge = RidgeCV()
2 modelRidge.fit(X,y)
3 print('岭回归剔除的变量:%d'%sum(modelRidge.coef_ == 0))
4 print('岭回归最优a: ', modelRidge.alpha_)
5 print('岭回归的误差: %.2f'%(1-modelRidge.score(X,y)))

```

岭回归剔除的变量: 0
岭回归最优a: 10.0
岭回归的误差: 0.02



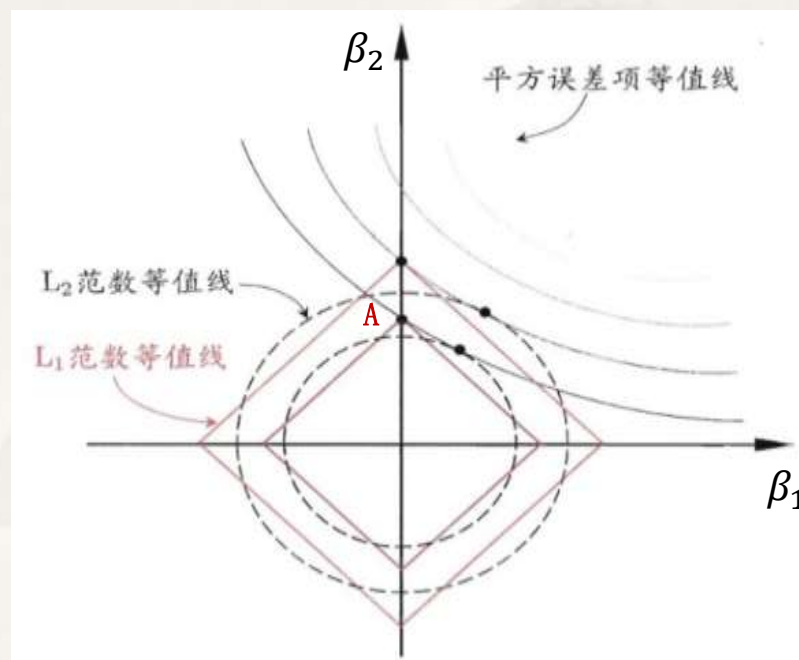
- Lasso中增大 α 会使更多系数被压缩到严格为零——**稀疏性**
- 岭回归中增大 α 会使系数趋近于零但通常不为零
- Lasso通过稀疏化，可在较低复杂度下达到较小的训练误差，而岭回归则需更多非零参数方能达到

外圈约束参数大，对应较小的收缩参数 α ，即对复杂度的惩罚小

- 岭回归和Lasso回归的比较:L2正则化(岭回归)还是L1正则化(Lasso回归)?

$$\hat{\beta} = \min_{\beta} \left[\frac{1}{2N} \sum_{i=1}^N \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j X_{ij} \right)^2 + \alpha \sum_{j=1}^p \beta_j^2 \right]$$

$$\hat{\beta} = \min_{\beta} \left[\frac{1}{2N} \sum_{i=1}^N \left(y_i - \beta_0 + \sum_{j=1}^p \beta_j X_{ij} \right)^2 + \alpha \sum_{j=1}^p |\beta_j| \right]$$

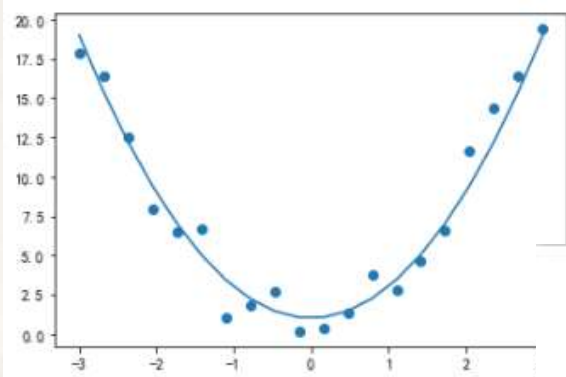


- 最优解落在坐标轴上得到简单模型
 - 如A点
- L1正则化较L2正则化有更多的机会将 β_j 严格约束为0
 - 如A点
- 从特征选择角度应倾向采用L1正则化, $\beta_j = 0$ 的变量 X_j 是不重要的变量
 - Lasso回归是嵌入策略下特征选择的优秀方法, 可同时实现预测建模

- 岭回归和Lasso回归的比较:L2正则化(岭回归)还是L1正则化(Lasso回归)?

- 数据模拟:

```
N = 20
x = np.linspace(-3, 3, N).reshape(N, 1)
np.random.seed(123)
u = np.random.normal(loc=0.0, scale=1.0, size=N)
fx = 1 + 2 * pow(x.ravel(), 2)
y = fx + u
plt.scatter(x, y)
plt.plot(x, fx)
```

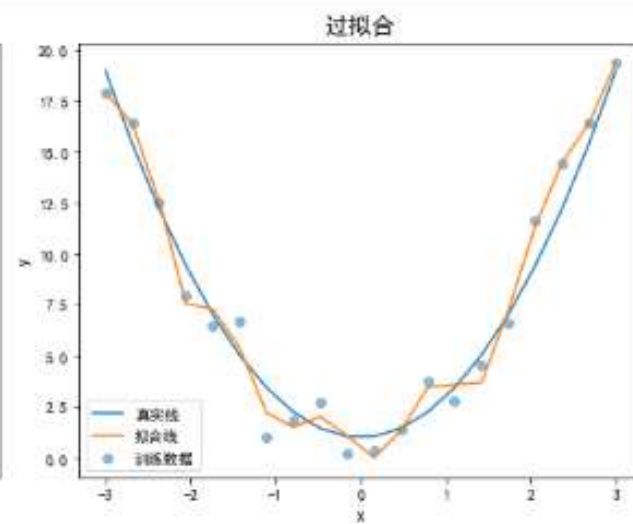
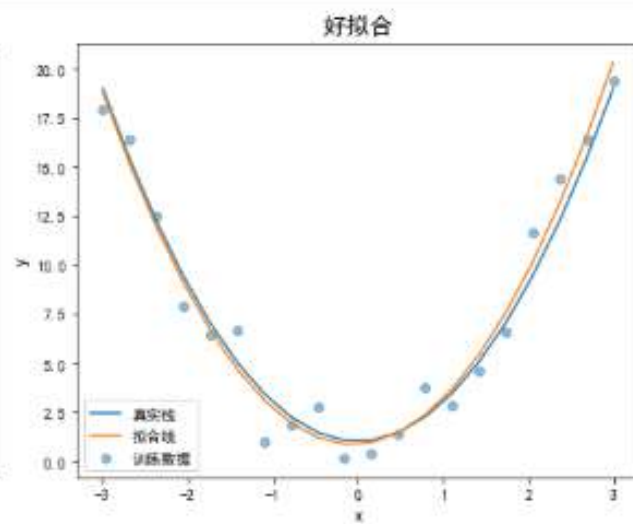
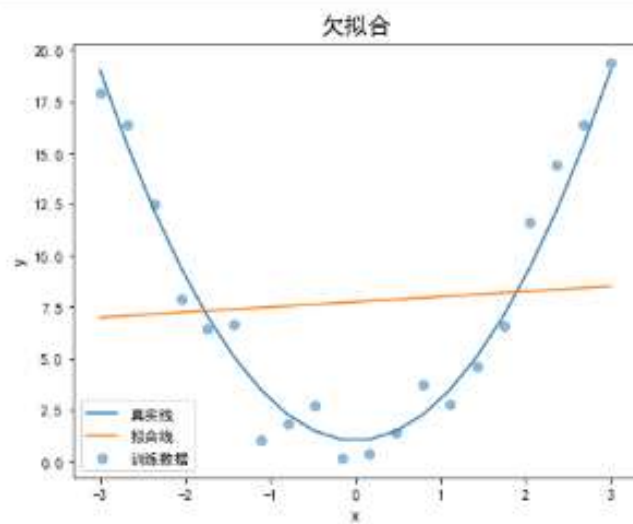


```
models = {'欠拟合': Pipeline([('X1', PolynomialFeatures(degree = 1)), ('modeling1', LM.LinearRegression())]),
          "好拟合": Pipeline([('X2', PolynomialFeatures(degree = 2)), ('modeling2', LM.LinearRegression())]),
          "过拟合": Pipeline([('X3', PolynomialFeatures(degree = 15)), ('modeling3', LM.LinearRegression())])
}
```

```
1 temp = np.arange(2, 5).reshape(3, 1)
2 poly = PolynomialFeatures(degree = 2)
3 poly.fit_transform(temp)
```

```
array([[ 1.,  2.,  4.],
       [ 1.,  3.,  9.],
       [ 1.,  4., 16.]])
```

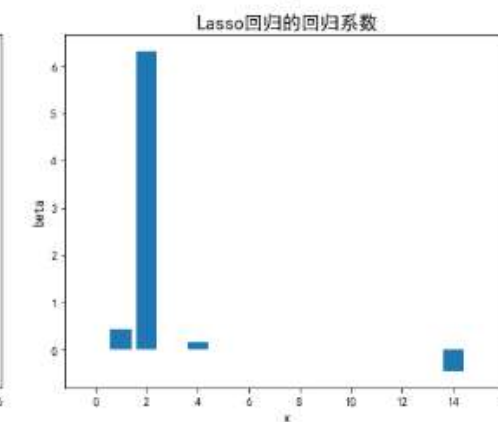
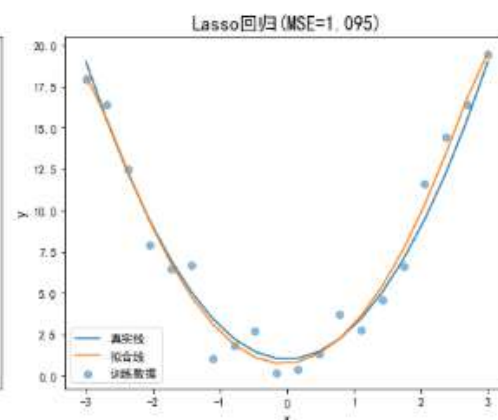
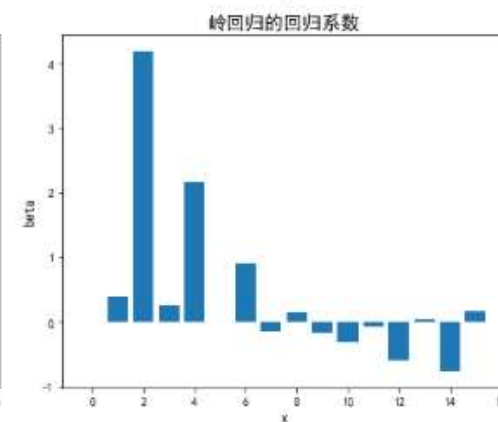
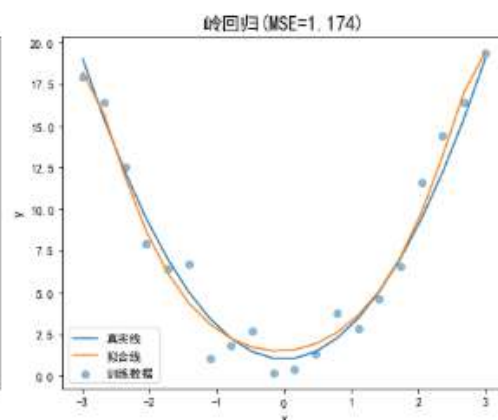
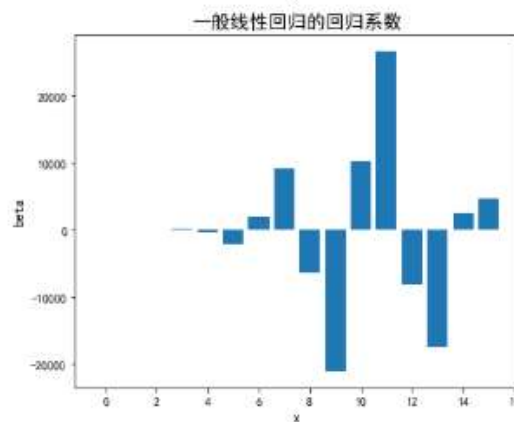
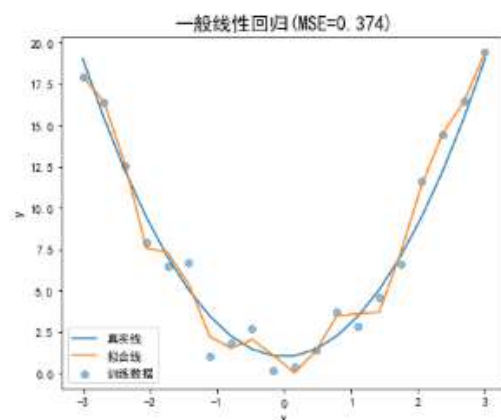
```
fig, axes = plt.subplots(1, 3, figsize = (18, 5))
for ax, (title, model) in zip(axes, models.items()):
    model.fit(x, y)
    y_hat = model.predict(x)
    ax.scatter(x, y, alpha = 0.5, label = '训练数据')
    ax.plot(x, fx, label = '真实线')
    ax.plot(x, y_hat, label = '拟合线')
```



- 岭回归和Lasso回归的比较:L2正则化(岭回归)还是L1正则化(Lasso回归)?
 - 数据模拟:

```
models = {'一般线性回归': make_pipeline(PolynomialFeatures(degree = 15), StandardScaler(), LM.LinearRegression()),  
         "岭回归": make_pipeline(PolynomialFeatures(degree = 15), StandardScaler(), RidgeCV()),  
         "Lasso回归": make_pipeline(PolynomialFeatures(degree = 15), StandardScaler(), LassoCV())  
}
```

```
fig, axes = plt.subplots(2, 3, figsize = (18, 10))  
for i, (title, model) in zip(np.arange(0, 3), models.items()):  
    model.fit(x, y)  
    y_hat = model.predict(x)  
    ax = axes[0, i]  
    ax.scatter(x, y, alpha = 0.5, label = '训练数据')  
    ax.plot(x, fx, label = '真实线')  
    ax.plot(x, y_hat, label = '拟合线')  
    ax.legend()  
    mse = round(mean_squared_error(y, y_hat), 3)  
    ax.set_title(f'{title} (MSE={mse})', fontsize = 16)  
    ax.set_xlabel('x', fontsize = 12)  
    ax.set_ylabel('y', fontsize = 12)  
  
    coefs = model.named_steps[model.steps[-1][0]].coef_  
    ax = axes[1, i]  
    ax.bar(np.arange(0, 16), coefs)  
    ax.set_title(f'{title}的回归系数', fontsize = 16)  
    ax.set_xlabel('x', fontsize = 12)  
    ax.set_ylabel('beta', fontsize = 12)
```



- 通过正则化防止过拟合问题!

- 案例七的分析：建立带正则项的二分类预测模型

- Logistic回归模型和参数估计 $L(y_i, \hat{P}(X_i)) = -[y_i \log \hat{P}(X_i) + (1 - y_i) \log (1 - \hat{P}(X_i))]$

- 最小损失函数(二项交叉熵损失)下的参数估计： $P(Y_i = 1 | X_i) = \frac{\exp(\mathbf{X}_i^T \boldsymbol{\beta} + \beta_0)}{1 + \exp(\mathbf{X}_i^T \boldsymbol{\beta} + \beta_0)}$

$$\hat{\boldsymbol{\beta}} = \operatorname{argmin}_{\boldsymbol{\beta}} \left\{ - \sum_{i=1}^N [Y_i (\mathbf{X}_i^T \boldsymbol{\beta} + \beta_0) - \ln(1 + \exp(\mathbf{X}_i^T \boldsymbol{\beta} + \beta_0))] \right\}$$

- 在Logistic回归的基础上，增加正则化项，求两者之和最小下的模型参数：

$$\hat{\boldsymbol{\beta}}(\alpha) = \operatorname{argmin}_{\boldsymbol{\beta}} \left\{ - \sum_{i=1}^N [Y_i (\mathbf{X}_i^T \boldsymbol{\beta} + \beta_0) - \ln(1 + \exp(\mathbf{X}_i^T \boldsymbol{\beta} + \beta_0))] + \alpha \sum_{j=1}^p \beta_j^2 \right\}$$

$$\hat{\boldsymbol{\beta}}(\alpha) = \operatorname{argmin}_{\boldsymbol{\beta}} \left\{ - \sum_{i=1}^N [Y_i (\mathbf{X}_i^T \boldsymbol{\beta} + \beta_0) - \ln(1 + \exp(\mathbf{X}_i^T \boldsymbol{\beta} + \beta_0))] + \alpha \sum_{j=1}^p |\beta_j| \right\}$$

• 案例七的分析：建立带正则项的分类预测模型：Lasso-Logistic

```

1 #LM.LogisticRegression(penalty='l1', solver='saga', C=1, random_state = 123)
2 modelLasso = LM.LogisticRegressionCV(penalty='l1', solver='saga', cv = 5, random_state = 123)
3 modelLasso.fit(X_train, y_train)
4 print(f'最优正则化参数: {modelLasso.C_} \n Lasso系数压缩至0的变量个数: {sum(modelLasso.coef_ == 0)}')
5 selector = SelectFromModel(estimator = modelLasso)
6 selector.fit(X_train, y_train)
7 print('保留的特征个数: %d' % len(selector.get_support(indices = True)))
8 Xtmp = selector.inverse_transform(selector.transform(X_train))
9 plt.figure(figsize = (5,5), dpi = 150)
10 for i, item in enumerate(ids, start = 1):
11     img = np.array(Xtmp[item, :]).reshape((16, 16))
12     plt.subplot(5, 5, i)
13     plt.imshow(img, cmap = cm.gray)

```

最优正则化参数: [21.5443469]
 Lasso系数压缩至0的变量个数: 44
 保留的特征个数: 203

selector共剔除了53个输入变量，其中44个系数等于0。
 因模型带有L1/L2范数约束时默认剔除阈值为1e-05

```

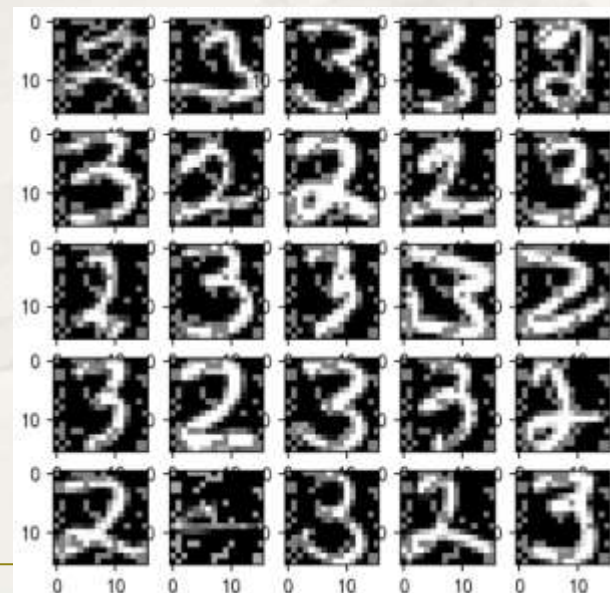
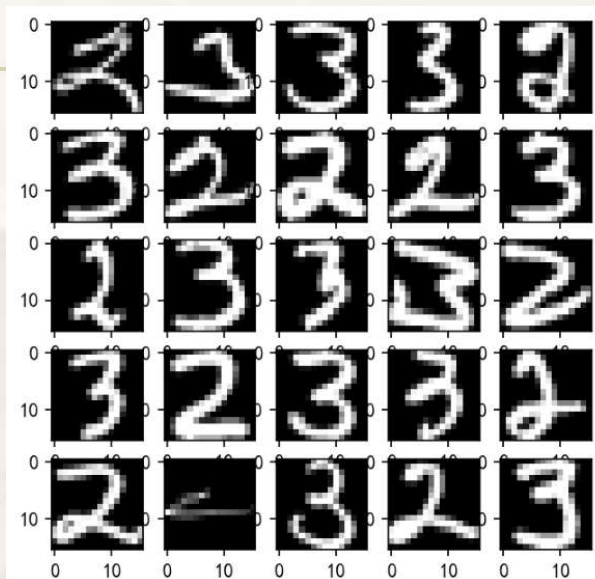
1 print('lasso-logistic的训练误差: %.2f' % (1 - modelLasso.score(X_train, y_train)))
2 print('模型评价 (测试集): \n', classification_report(y_test, modelLasso.predict(X_test)))

```

lasso-logistic的训练误差: 0.00

模型评价 (测试集):

	precision	recall	f1-score	support
2.0	0.98	0.98	0.98	219
3.0	0.98	0.98	0.98	198
accuracy			0.98	417
macro avg	0.98	0.98	0.98	417
weighted avg	0.98	0.98	0.98	417



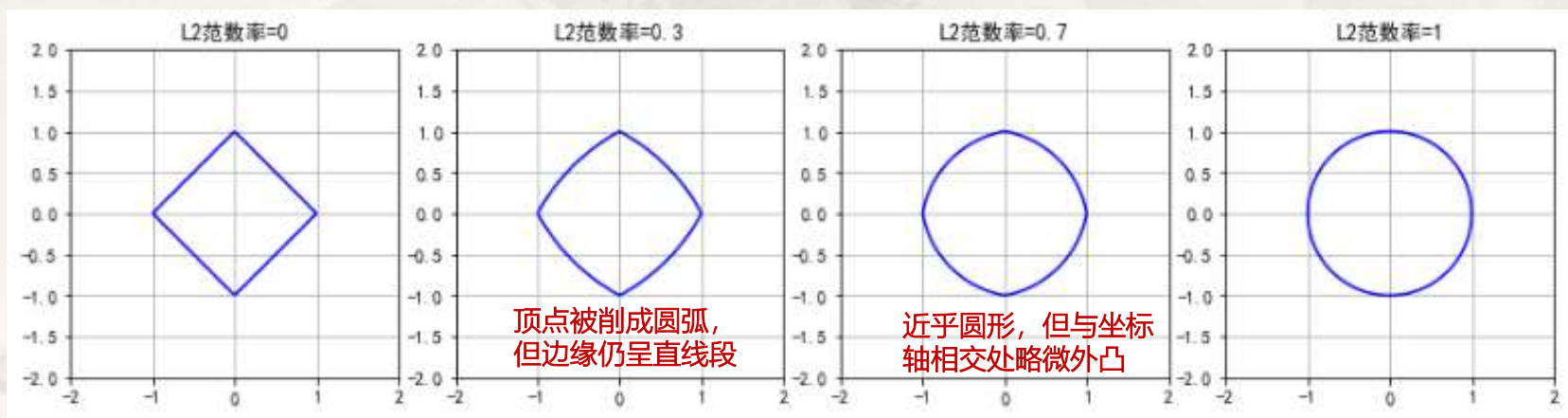
弹性网(Elastic Net)回归

- Lasso回归和岭回归的结合，同时引入L1正则化和L2正则化

- 目标函数：

$$\sum_{i=1}^N (y_i - \beta_0 - \sum_{j=1}^p \beta_j X_{ij})^2 + \alpha \sum_{j=1}^p (\gamma \beta_j^2 + (1 - \gamma) |\beta_j|)$$

- $0 \leq \gamma \leq 1$ ， $\gamma = 0$ 时为Lasso回归， $\gamma = 1$ 时为岭回归
- 该系数控制岭回归和Lasso回归的“贡献”率，称为L2范数率



- 不同 L2 范数率下弹性网的约束区域图形是一个连续变形过程
 - 从 Lasso 的尖角菱形逐渐变圆, 最终成为岭回归的圆形
- 这种几何形状使其兼具稀疏性与群组效应

- 具有Lasso回归特点可将不重要的输入变量系数精确压缩为零——稀疏性
- 具有岭回归特点可将高度相关变量(系数相近)同时选入(或剔除)模型——群组效应(稳定识别相关特征组)
- L2惩罚的连续收缩使得系数估计更加平滑，对噪声的鲁棒性更好，避免Lasso在某些情况下系数估计的过大波动

弹性网(Elastic Net)回归

- Python中，弹性网回归的目标函数为：

$$\frac{1}{2N} \sum_{i=1}^N (y_i - \beta_0 - \sum_{j=1}^p \beta_j X_{ij})^2 + \frac{\alpha(1-\gamma)}{2} \sum_{j=1}^p \beta_j^2 + \alpha\gamma \sum_{j=1}^p |\beta_j|$$

$\gamma = 0$ 时为岭回归， $\gamma = 1$ 时为Lasso回归，并称 γ 为**L1范数率**

• 弹性网回归: 探讨L1范数率对回归系数的影响

Chapter10-5.ipynb

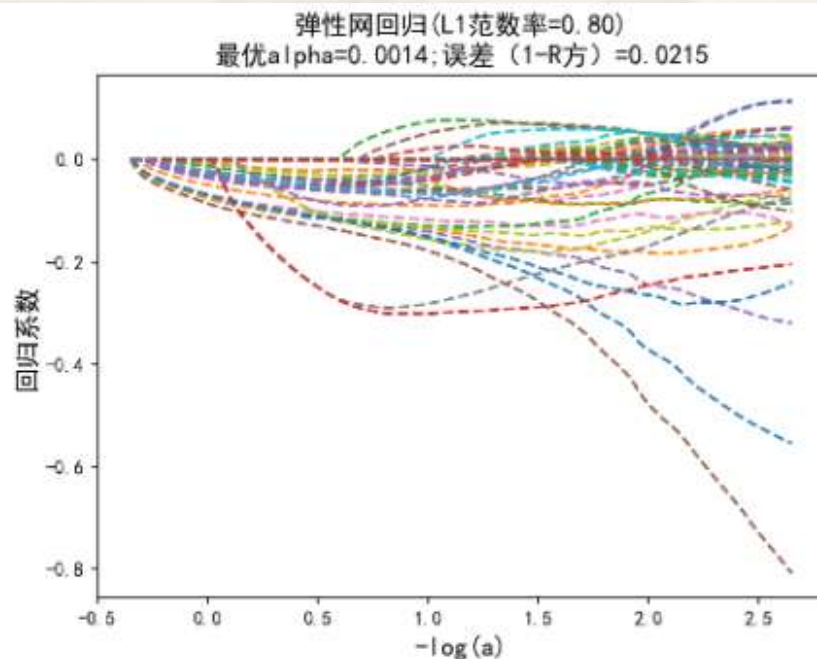
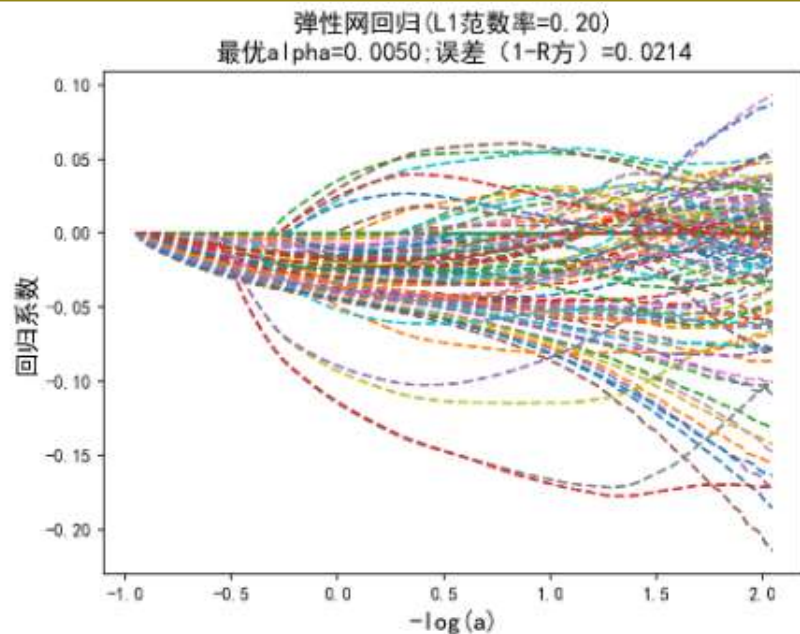
```
1 data = pd.read_table('特征提取.txt', sep = ',', header = None)
2 print(f'数据集的形状: {np.shape(data)}')
3 data
```

数据集的形状: (1663, 257)

	0	1	2	3	4	5	6	7	8	9	...
0	2.989144	-1.0	-1.0	-1.0	-1.000	-1.000	-0.928	-0.204	0.751	0.466	...
1	3.009973	-1.0	-1.0	-1.0	-0.830	0.442	1.000	1.000	0.479	-0.328	...
2	1.002830	-1.0	-1.0	-1.0	-1.000	-1.000	-1.000	-1.000	0.510	-0.213	...
3	0.984937	-1.0	-1.0	-1.0	-1.000	-1.000	-1.000	-1.000	-0.909	0.801	...
4	0.994214	-1.0	-1.0	-1.0	-1.000	-1.000	-1.000	-0.998	0.613	-0.385	...

```
1 X = data.iloc[:,1:-1]; y = data.iloc[:,0]
2 fig, axes = plt.subplots(nrows = 1, ncols = 2, figsize = (15, 5), dpi = 150)
3 ratios = [0.2, 0.8]
4 for i, ratio in enumerate(ratios):
5     alphas_enet, coefs_enet, _ =enet_path(X, y, l1_ratio=ratio)
6     axes[i].plot(-np.log10(alphas_enet), coefs_enet.T, linestyle = '--')
7     model = ElasticNetCV(l1_ratio = ratio)
8     model.fit(X, y)
9     axes[i].set_title('弹性网回归 (L1范数率=%.2f)\n最优alpha=%.4f; 误差 (1-R方) =%.4f'%
10                       (ratio, model.alpha_, 1-model.score(X, y)), fontsize = 14)
11     axes[i].set_xlabel('-log(a)', fontsize = 14)
12     axes[i].set_ylabel('回归系数', fontsize = 14)
13     axes[i].axis('tight')
```

收缩参数 α 默认从大到小取100个值



• 案例七的分析：建立基于弹性网的分类预测模型：Elasticnet-Logistic

```
1 modelEnet = LM.LogisticRegressionCV(penalty='elasticnet', solver='saga', l1_ratios=[0.2, 0.8], cv = 5, random_state = 123)
2 modelEnet.fit(X_train, y_train)
3 print(f'最优L1范数率: {modelEnet.l1_ratio_}; 最优正则化参数: {modelEnet.C_}')
4 print(f'系数压缩至0的变量个数: {sum(modelEnet.coef_ == 0)}')
5 selector = SelectFromModel(estimator = modelEnet)
6 selector.fit(X_train, y_train)
7 print(f'保留特征数={len(selector.get_support(indices = True))}')
8 Xtmp = selector.inverse_transform(selector.transform(X_train))
9 plt.figure(figsize = (5,5), dpi = 150)
10 for i, item in enumerate(ids, start = 1):
11     img = np.array(Xtmp[item, :]).reshape((16, 16))
12     plt.subplot(5, 5, i)
13     plt.imshow(img, cmap = cm.gray)
14 plt.show()
```

Chapter10-5.ipynb

最优L1范数率: [0.8]; 最优正则化参数: [21.5443469]
系数压缩至0的变量个数: 27
保留特征数=89

selector共剔除了167个输入变量，其中27个系数等于0。
因模型带有L1/L2范数约束时默认剔除阈值为1e-05

```
1 print('弹性网络训练误差: %.2f'%(1 - modelEnet.score(X_train, y_train)))
2 print('模型评价 (测试集): \n', classification_report(y_test, modelEnet.predict(X_test)))
```

弹性网络训练误差: 0.00
模型评价 (测试集):

	precision	recall	f1-score	support
2.0	0.98	0.98	0.98	219
3.0	0.98	0.98	0.98	198
accuracy			0.98	417
macro avg	0.98	0.98	0.98	417
weighted avg	0.98	0.98	0.98	417

本模型的复杂度高于
Lasso-Logistic

